README.md | agenda.md

EXPLORER

WORKSPACE

- .github
  - agents
  - instructions
  - prompts
  - memory
  - skills
- README.md

Stop Prompting,
Start Building Agents
with Criteria

// Building smarter agent architectures in GitHub Copilot + VS Code

LM **Lucía Muñoz**
Power Platform Consultant

SA **Samantha Cruz**
Data Engineer Team Lead

// iSolutions · Tech Meetup · 2026

Who we are

and why are we here

Samantha Cruz

Data Engineering Team Lead

Lucia Munoz Salas

Power Platform AI Developer

EXPLORER

AGENT PROJECT

- agents
- instructions
  - global.instructions.md
  - repo.instructions.md
- memory
- prompts
- skills
  - SKILL.md
  - scripts

M↓ 01_layers_and_approvals.md

Instructions, Skills and Approvals

Three ways to teach your agent how you work



Global Instructions

Your preferences, always on



Repo Instructions

This project's naming, architecture and conventions



Skills

Loaded only when the task needs them

Approvals: how much it can do without you



Need Approval

Every action waits for you. Slowest, safest



Bypass Approvals

Actions with low risk skip confirmation



Auto / Autopilot

Agent approves itself, runs until done or blocked

Don't Just Use a Public Skill, Make it Yours

Fork it, don't copy it

A public skill is a starting point, not the finished product

Inject real project context

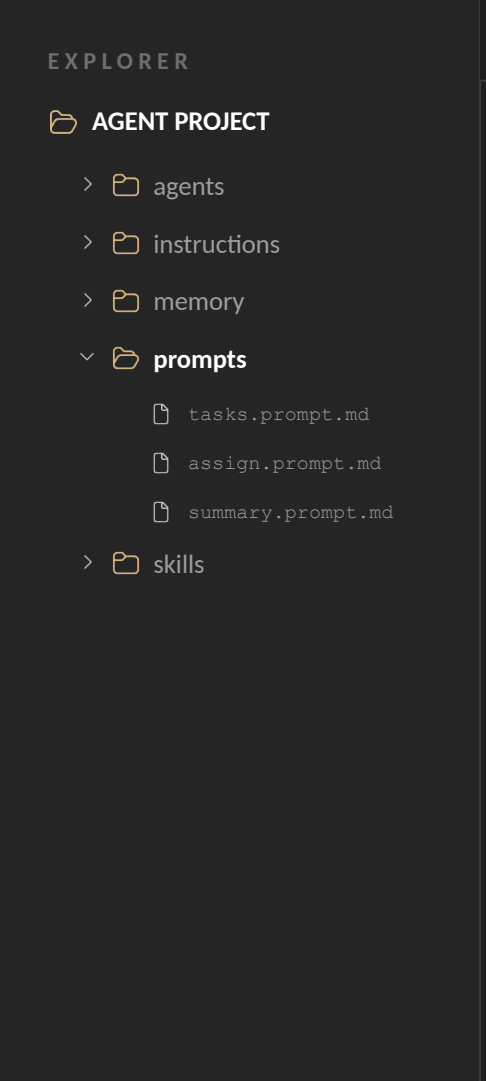
Your repo's naming, your layers, your conventions. Not generic best practices

Make it improve itself

Add an instruction inside the skill: if something goes wrong, propose an update to this file

Use the skill → Something fails → Improve the skill

Every mistake becomes a permanent improvement, not something you fix once and forget



03_prompt_files.md

Prompt Files: Stop Rewriting, Start Reusing

- Prompt files live in .github/prompts/ as .prompt.md files
- Triggered with /filename directly in VS Code chat
- One file per recurring workflow

EXPLORER

AGENT PROJECT

- agents
- instructions
- memory
- prompts
  - tasks.prompt.md
  - assign.prompt.md
  - summary.prompt.md
- skills

M↓ 03_prompt_files.md

Prompt Files: Stop Rewriting, Start Reusing

Example: 3 prompt files, a full development cycle

1



Prioritize tasks

Connects to Azure DevOps and orders open tasks by priority

2



Assign to the right agent

Picks the best fit agent, splits oversized tasks

3



Summarize and update

Drafts a summary and updates Azure DevOps

EXPLORER

AGENT PROJECT

- agents
- instructions
- memory
  - glossary.md
  - error_log.md
  - repo_summary.md
- prompts
- skills

M↓ 04_memory_files.md

Memory Files: Give Your Agent Real Context

Plain .md files, referenced in agent instructions, not loaded automatically



Glossary

Keeps terminology consistent across the whole project, session after session



Error log

Agents write directly to it. You review it at the end of each iteration



Repo summary

A map of the codebase, so the agent navigates without reading everything

How Agents Consume Credits

It's not one question. It's every token in between

GitHub Copilot now bills by **token consumption**, effective since June 2026

Input tokens

Your message, files, context

Output tokens

Everything the model generates

Cached tokens

Cheaper, but they still count

Agent reads 10 files before answering





10 files worth of input tokens

Bigger context means a more expensive bill on every single response

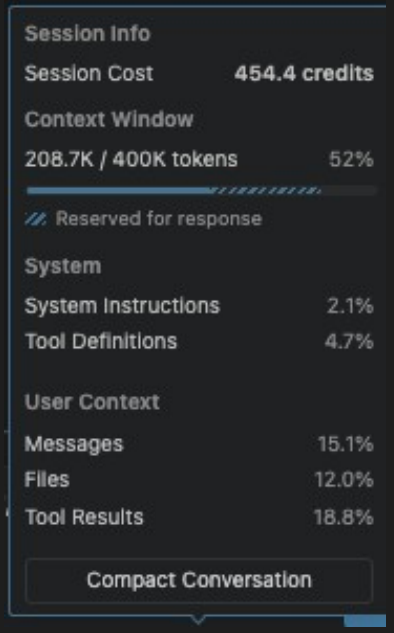
EXPLORER

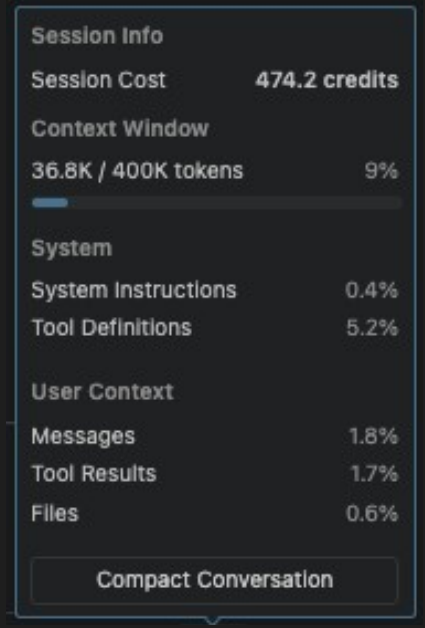
AGENT PROJECT

- agents
- instructions
- memory
- prompts
- skills

M↓ 05_credits.md

How Agents Consume Credits





EXPLORER

AGENT PROJECT

- agents
- instructions
- memory
- prompts
- skills

M↓ 05_credits.md

How Agents Consume Credits



Copilot Insights

Emanuele Bartolesi | 6,095 | ★★★★★ (8)

♥ Sponsor

Understand your GitHub Copilot plan, quotas, and limits at a glance

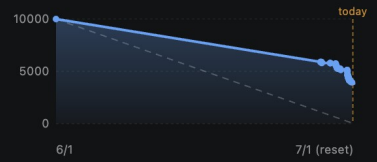
Disable | Uninstall | ☑ Auto Update ⚙

Sprint Burn-down

Since last refresh: **-25 credits**

Since yesterday: **-1205 credits**

AI Credits Burn-down



Actual — Ideal --- Trend

On track · 3913 credits ahead of pace

Weighted Prediction

Predicted Daily Usage High Accuracy

237 credits/day

Est. daily cost:	~$2.37/day
Projected monthly cost:	~$71.10
Est. days until exhausted:	16 days
Sustainability:	✓ On track for reset

Based on 11 data points from local history

EXPLORER

AGENT PROJECT

- agents
- instructions
- memory
- prompts
- skills

M↓ 06_choosing_model.md

Choosing the Right Model

Two axes: how well it reasons, and what it costs per interaction

Reasoning ↑

Fast / cheap Routine edits, boilerplate, autocomplete	**Deep reasoning** Hard debugging, architecture, real ambiguity
General purpose Your default when you're not sure	**Higher multiplier** Big refactors, use it on purpose

Cost →

How to decide in practice

Most daily tasks don't need the most expensive model. That's a requirements decision, not a quality one

A 3x multiplier costs three times what a 0.33x multiplier costs for the same interaction

Auto mode picks for you based on the task, with about a 10% discount on top

```
1 AI credit = $0.01 USD
```

EXPLORER

AGENT PROJECT

- agents
- instructions
- memory
  - glossary.md
  - error_log.md
  - repo_summary.md
- prompts
- skills

M↓ 07_stay_in_control.md

How to Keep it Under Control

Work smarter, not wider



Repo summary

Agents go straight to what they need. No exploring, no scanning, no accumulating unnecessary context. One file read instead of ten



Memory files

Terminology, error patterns, decisions: every piece of structured knowledge given upfront replaces an expensive exploration with a cheap, targeted read

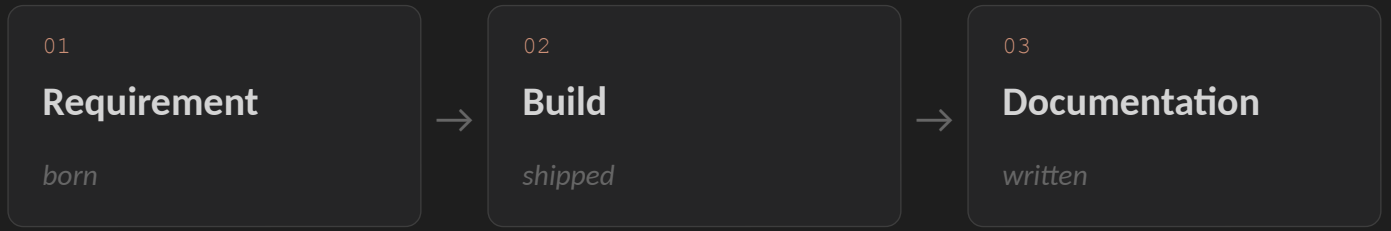
A focused agent is a cheap agent

Use Case – Agent Development Team

How to Actually use AI as development assistant

AI-First Approach

// "We don't use AI as a tool — we integrate it as part of the team."



// From the moment a requirement is born · to the moment what was done gets documented.

// end-to-end process

From customer input to deployed solution.

// Five phases, specialised agents, one continuous loop.

01 INPUTS
Customer story & requirements
Policies, processes, UX guidelines, acceptance criteria.

→

02 PLANNING
Specification agent
Translates requirements into an actionable design.

→

03 IMPLEMENTATION
Specialised build agents
Data model · Forms & views · Plugins · Flows.

→

04 ARTIFACTS
Solution components
Metadata, scripts, plugins, cloud flows, configuration.

→

05 RECONCILIATION
Documentation agent
Closes the loop — runbooks and as-built reconciliation.

INTEGRATION LAYER

MCP servers connect agents to the real tools.

MCP
Azure DevOps

MCP
Dataverse Metadata

// Specification agent designs · build agents implement · documentation agent reconciles.

features.md | pipeline.md

EXPLORER

WORKSPACE

- .github
  - agents
  - instructions
  - prompts
  - memory
  - skills
- README.md

```
// what makes it different
```

Two features that make it robust.

01

Living documentation

// Every action writes itself to the wiki.

Documentation updates itself and feeds back into the agents as context, so each next decision is better-informed. A self-reinforcing loop.

02

Shared error memory

// No mistake is ever repeated twice.

When something fails and gets resolved by an agent or a developer's correction, the solution is recorded. Every agent learns. In a distributed team, that's worth a lot.

// A self-reinforcing loop · shared learning across the team.